Duplicate Pull Requests from Coding Agents: A Live-Queue Evaluation of 66 Detection Strategies

[TODO: authors]
[TODO: affiliations]

***Abstract*—Coding agents now open pull requests (PRs) at scale, and many of them duplicate work that is already open: in one hand-labelled sample, 23.7% of failed agent PRs were duplicates [1]. Duplicate-PR detectors were built and evaluated on human PRs, scored against random pairs of PRs. We ask how well they work on agent PRs when scored the way they are used: at the moment a new PR is opened, against the PRs actually open in that repository (a *live queue*). On AIDev [2] we build 1867 live-queue duplicate pairs (342 between different authors) and 2965 PRs with no duplicate, and compare 66 strategies: lexical, code-overlap, link, embedding, existing duplicate-PR bots, learned combinations, cross-encoders, and LLM judges (Qwen2.5 1.5B–14B, Claude Haiku 4.5). A cheap learned combination of text, code, link, and timing signals (Jinx) ranks the true duplicate first for 0.901 of cross-author pairs versus 0.810 for the INTRUDE-style detector and 0.778 for the best existing bot, with 2.5% versus 4.6% false alarms. No LLM judge or re-ranker improves on it, and none of the 65 strategies beats the classic detector on Recall@5 with a confidence interval above zero. Random-pair scoring ranks strategies differently from live-queue scoring, and duplicates that share no files remain unsolved.**

I. Introduction

[TODO: Motivation: agent PR volume; duplicates in failure studies [1], [3], [4], [5]; maintainers' cost.] [TODO: Gap: no detector evaluated on agent PRs; random-pair negatives [6]; analogy to bug reports [7].] [TODO: Contributions: (1) live-queue benchmark on AIDev; (2) 66-strategy comparison incl. LLM judges; (3) duplicate taxonomy and per-class results; (4) evidence that random-pair scoring misleads; (5) open tool and artifact.]

II. Research Questions

- **RQ1** How well do existing duplicate-PR detectors and bots work on agent PRs under live-queue scoring?
- **RQ2** Do embeddings, cross-encoders, or LLM judges beat cheap combined signals?
- **RQ3** Which kinds of duplicates (taxonomy) remain hard?
- **RQ4** Does random-pair scoring rank strategies the same way as live-queue scoring?

III. Benchmark

[TODO: AIDev v5 (CC BY 4.0, cutoff 31 Mar 2026); silver labels from "duplicate of / superseded by / in favor of #N" comments; live queue = PRs open in the repo when the new PR was created; 76,209 PRs; median queue 40, p90 314; 1525 self-replacement, 342 cross-author (228 same agent, 114 cross-agent); 2965 no-duplicate queries.] [TODO: Taxonomy: A code twin, B partial overlap, C same files/different code, D no shared files; same issue/different code; same code/different description.] [TODO: Label audit result (precision of silver labels) – pending.]

TABLE I
Cross-author pairs ($n = 342$). False alarms on 2965 PRs with no duplicate.

Strategy	R@1	R@5	MRR	False alarms % [95% CI]
Jinx (ours)	0.901	0.971	0.932	2.5 [1.9, 3.3]
All signals, LR	0.906	0.974	0.935	1.6 [1.1, 2.1]
Jinx + BGE rerank + Qwen-7B	0.901	0.974	0.931	2.8 [2.2, 3.5]
Claude Haiku 4.5 + Jinx	0.898	0.971	0.932	1.8 [1.3, 2.3]
Qwen2.5-14B + Jinx	0.871	0.971	0.917	2.9 [2.3, 3.7]
Claude Haiku 4.5 alone	0.842	0.971	0.902	5.4 [4.6, 6.1]
INTRUDE + time [6], [8]	0.810	0.953	0.873	4.6 [3.7, 5.7]
contrib-guard (bot)	0.778	0.918	0.843	11.7 [8.9, 14.6]
Links only	0.839	0.924	0.882	3.8 [3.0, 4.8]
TF-IDF [9]	0.757	0.921	0.832	9.4 [7.8, 10.9]
BGE-small embedding	0.763	0.915	0.830	6.4 [5.3, 7.5]

TABLE II
Recall@1 per relation and duplicate class (cross-author unless noted).

Group (Recall@1)	n	Jinx	All LR	INTRUDE+t	c-guard	Haiku+J
same agent	228	0.899	0.908	0.825	0.811	0.895
cross-agent	114	0.904	0.904	0.781	0.711	0.904
A: code twin	139	0.971	0.978	0.957	0.892	0.978
B: partial overlap	164	0.896	0.909	0.762	0.720	0.878
C: same files	10	0.700	0.600	0.400	0.500	0.800
D: no shared files	29	0.655	0.655	0.517	0.655	0.655
no link at all	140	0.821	0.829	0.643	0.664	0.829
self-replacement	1525	0.887	0.892	0.857	0.782	–

IV. Strategies

[TODO: Families and exact definitions; re-implemented bots and what was run as-is; training protocol: repository-grouped 5-fold CV.]

V. Evaluation Protocol

[TODO: Recall@1/5, MRR; false-alarm rate at a threshold fixed on training folds; repository-cluster bootstrap (2000 resamples); paired differences; pre-registered kill rule.]

VI. Results

RQ1. Against the INTRUDE-style detector, Jinx improves cross-author Recall@1 by +9.1 points [+5.0, +13.1] and MRR by +5.9 [+2.7, +9.3], but Recall@5 only by +1.8 [-0.9, +5.2]: the pre-registered Recall@5 criterion is not met. Against the best existing bot, Recall@5 improves by +5.3 [+1.2, +9.7]. **RQ2.** [TODO: LLM judges and rerankers: no CI-backed gain over Jinx; cost and latency.] **RQ3.** [TODO: Classes C and D; no-link pairs; same code/different description.] **RQ4.**

[TODO: Kendall τ between random-pair AUC and live-queue Recall@1; strategies whose rank flips.]

VII. Threats to Validity

[TODO: Silver labels; link bias (links-only rule reaches 0.839 Recall@1); no agent–human pairs; re-implemented bots; single dataset.]

VIII. Related Work

Classic duplicate-PR detection [9], [10], [6], [8], [11], [12], [13], [14]; competing and redundant PRs [15], [16]; agent-PR studies [1], [17], [18], [19]; duplicate bug reports [7], [20]. [TODO: Commercial review tools and open bots (incl. doppelgänger, DupeHound).]

IX. Conclusion

[TODO:]

Data Availability

[TODO: Zenodo DOI for benchmark, code, and raw results.]

References

[1] R. Ehsani *et al.*, "Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub," 2026, mSR 2026.

[2] H. Li *et al.*, "AIDev: Studying AI coding agents on GitHub," 2026.

[3] Alam, Mondal, and C. K. Roy, "Why are AI agent involved pull requests (fix-related) remain unmerged?" 2026.

[4] Nakashima *et al.*, "Why agentic-PRs get rejected: A comparative study of coding agents," 2026.

[5] Abujadallah, Arabat, and M. Sayagh, "Understanding the rejection of fixes generated by agentic pull requests: Insights from the AIDev dataset," 2026, tODO: verify author first names.

[6] L. Ren, S. Zhou, C. Kästner, and A. Wąsowski, "Identifying redundancies in fork-based development," in *Proc. SANER*, 2019.

[7] T. Zhang, D. Han, V. Vinayakarao, I. C. Irsan, B. Xu, F. Thung, D. Lo, and L. Jiang, "Duplicate bug report detection: How far are we?" *ACM Trans. Softw. Eng. Methodol.*, 2023.

[8] Q. Wang, B. Xu, X. Xia, T. Wang, and S. Li, "Duplicate pull request detection: When time matters," in *Proc. Internetware*, 2019.

[9] Z. Li, G. Yin, Y. Yu, T. Wang, and H. Wang, "Detecting duplicate pull-requests in GitHub," in *Proc. Internetware*, 2017.

[10] Y. Yu, Z. Li, G. Yin, T. Wang, and H. Wang, "A dataset of duplicate pull-requests in GitHub," in *Proc. MSR*, 2018.

[11] J. Jiang, Q. Li, J. Guo, Z. Li, L. Zhang, and G. Li, "DupHunter: Detecting duplicate pull requests in fork-based development," *IEEE Trans. Softw. Eng.*, 2023.

[12] Z. Liao, Y. Liu, Y. Lan, and Y. Sun, "DupLLM: Duplicate pull requests detection based on large language model," in *Proc. APSEC*, 2024.

[13] R. Galbraith and D. Greer, "Detecting duplicate effort in GitHub contributions," in *Proc. ENASE*, 2025.

[14] B. Chekaya, Garrouch, and Omri, "Duplicate pull requests in code management platforms: A systematic literature review," Research Square preprint, 2026, tODO: verify author first names.

[15] X. Zhang, Y. Chen, Y. Gu, W. Zou, X. Xie, and X. Jia, "How do multiple pull requests change the same code: A study of competing pull requests in GitHub," in *Proc. ICSME*, 2018.

[16] Z. Li, Y. Yu, M. Zhou, T. Wang, and G. Yin, "Redundancy, context and preference: An empirical study of duplicate pull requests in OSS projects," *IEEE Trans. Softw. Eng.*, 2022.

[17] Xu *et al.*, "AI agent pull requests on GitHub: Frequency, structure and merge conflict rates," 2026.

[18] Sarkar, "Before the pull request: Mining multi-agent coordination," 2026.

[19] Khemissi *et al.*, "Humans integrate, agents fix: How agent-authored pull requests are referenced in practice," 2026.

[20] "CUPID: Leveraging ChatGPT for more accurate duplicate bug report detection," 2023, tODO: authors.